

Práctica 4: Problemas de Concurrency en T-SQL

Análisis de anomalías de lectura y control de transacciones sobre AdventureWorks2022

Materia: Bases de Datos Distribuidas **Base de datos:** AdventureWorks2022

Tecnología: Transact-SQL (T-SQL) **SGBD:** Microsoft SQL Server

Nota para realizar la práctica: se usan dos ventanas de consulta en SSMS, una para la Sesión A y otra para la Sesión B, para simular las transacciones concurrentes.

1. Lectura Sucia (Dirty Read / Dependencia no Confirmada)

Ocurre cuando una transacción lee datos modificados por otra transacción que todavía no ha realizado un COMMIT. Si la primera hace ROLLBACK, la segunda trabajó con un dato que no quedó guardado.

Sesión A (modificación sin confirmar)

```
USE AdventureWorks2022;
BEGIN TRANSACTION;

UPDATE Production.Product
SET SafetyStockLevel = 9999
WHERE ProductID = 1;

-- Mantiene el bloqueo exclusivo (X)
WAITFOR DELAY '00:00:08';

-- Cancela la modificación
ROLLBACK TRANSACTION;
```

Sesión B (lectura sucia)

```
USE AdventureWorks2022;

SET TRANSACTION ISOLATION LEVEL
READ UNCOMMITTED;

SELECT ProductID, Name, SafetyStockLevel
FROM Production.Product
WHERE ProductID = 1;

-- Lee SafetyStockLevel = 9999
-- ¡Dato inválido tras el ROLLBACK!
```

Conclusión: Con READ UNCOMMITTED, la Sesión B puede leer el valor 9999 aunque la Sesión A todavía no lo haya confirmado. Después A hace ROLLBACK, así que B leyó un valor que realmente no quedó en la tabla. Esto podría causar decisiones incorrectas en un sistema real.

2. Lectura No Repetible (Non-Repeatable Read / Análisis Inconsistente)

Pasa cuando una transacción lee el mismo registro dos veces y obtiene valores diferentes porque otra transacción lo modificó y confirmó el cambio entre ambas lecturas.

Sesión A (lectura doble)

```
USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL
READ COMMITTED; -- Nivel por defecto

BEGIN TRANSACTION;

-- Primera lectura (StandardCost = 13.08)
SELECT ProductID, StandardCost
FROM Production.Product WHERE ProductID = 707;

WAITFOR DELAY '00:00:08';

-- Segunda lectura dentro de la misma TX
SELECT ProductID, StandardCost
FROM Production.Product WHERE ProductID = 707;
COMMIT;
```

Sesión B (modificación intermedia)

```
USE AdventureWorks2022;

-- Ocurre durante los 8 segundos de espera
UPDATE Production.Product
SET StandardCost = 25.00
WHERE ProductID = 707;

-- La modificación se confirma inmediatamente
```

Conclusión: En READ COMMITTED los bloqueos compartidos se liberan al terminar el SELECT. Por eso B puede cambiar el costo y A termina viendo dos valores distintos en la misma transacción. Para evitarlo se requiere REPEATABLE READ.

3. Lectura Fantasma (Phantom Read)

Ocurre cuando una transacción hace dos veces una consulta de rango y en la segunda aparecen filas adicionales que otra transacción insertó y confirmó durante la espera.

Sesión A (conteo de rango)

```
USE AdventureWorks2022;
SET TRANSACTION ISOLATION LEVEL
REPEATABLE READ;

BEGIN TRANSACTION;
```

```
-- Consulta de rango inicial
SELECT COUNT(*) AS TotalProductos
FROM Production.Product
WHERE ListPrice BETWEEN 100 AND 150;

WAITFOR DELAY '00:00:08';

-- Segunda lectura: reaparecen nuevos registros
SELECT COUNT(*) AS TotalProductos
FROM Production.Product
WHERE ListPrice BETWEEN 100 AND 150;
COMMIT;
```

Sesión B (inserción en rango)

```
USE AdventureWorks2022;

-- Inserción dentro del rango evaluado
INSERT INTO Production.Product (
    Name, ProductNumber, SafetyStockLevel,
    ReorderPoint, StandardCost, ListPrice,
    DaysToManufacture, SellStartDate
) VALUES (
    'Bicicleta Test', 'BK-T999', 500,
    375, 80.00, 125.00, 1, GETDATE()
);
```

Conclusión: REPEATABLE READ protege los registros existentes, pero no evita que aparezcan filas nuevas dentro del rango. En este ejemplo B agrega un producto con precio 125.00. Para bloquear también las inserciones del rango se utiliza SERIALIZABLE.